

303105349 - Compiler Design

Course: BTech

Semester: 6

**Prerequisite:** Algorithms, Data Structures, Assembly Language Program, Theory of Computation, C/C++ Programming Skills | 203105205 - Data Structure and Algorithms

**Rationale :** Compiler Design is a fundamental subject of Computer Engineering. Compiler design principles provide an in-depth view of translation, optimization and compilation of the entire source program. It also focuses on various designs of compiler and structuring of various phases of compiler. It is inevitable to grasp the knowledge of various types of grammar, lexical analysis, yacc, FSM(Finite State Machines) and correlative concepts of languages.

| Teaching and Examination Scheme |                      |                 |                     |        |                    |    |   |                |   |       |
|---------------------------------|----------------------|-----------------|---------------------|--------|--------------------|----|---|----------------|---|-------|
| Teaching Scheme                 |                      |                 |                     |        | Examination Scheme |    |   |                |   | Total |
| Lecture<br>Hrs/Week             | Tutorial<br>Hrs/Week | Lab<br>Hrs/Week | Seminar<br>Hrs/Week | Credit | Internal Marks     |    |   | External Marks |   |       |
|                                 |                      |                 |                     |        | T                  | CE | P | T              | P |       |
| 3                               | 0                    | 0               | 0                   | 3      | 20                 | 20 | - | 60             | - | 100   |

SEE - Semester End Examination, T - Theory, P - Practical

| Course Content |                                                                                                                                                                                                                                                                                                                                                                                   | W - Weightage (%) , T - Teaching hours |   |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|---|
| Sr.            | Topics                                                                                                                                                                                                                                                                                                                                                                            | W                                      | T |
| 1              | <b>Overview of compilation :</b><br>The structure of a compiler and applications of compiler technology; Lexical analysis - The role of a lexical analyzer, specification of tokens, recognition of tokens, hand-written lexical analyzers, LEX, examples of LEX programs.                                                                                                        | 10                                     | 8 |
| 2              | <b>Introduction to syntax analysis</b><br>Role of a parser, use of context-free grammars (CFG) in the specification of the syntax of programming languages, techniques for writing grammars for programming languages (removal left recursion, etc.), non-context-free constructs in programming languages, parse trees and ambiguity, examples of programming language grammars. | 10                                     | 7 |
| 3              | <b>Top-down parsing</b><br>FIRST & FOLLOW sets, LL(1) conditions, predictive parsing, recursive descent parsing, error recovery. LR-parsing - Handle pruning, shift-reduce parsing, viable prefixes, valid items, LR(0) automaton, LR-parsing algorithm, SLR(1), LR(1), and LALR(1) parsing. YACC, error recovery with YACC and examples of YACC specifications.                  | 20                                     | 7 |



|              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |            |           |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------|-----------|
| <b>4</b>     | <b>Syntax-directed definitions (attribute grammars)</b><br>Synthesized and inherited attributes, examples of SDDs, evaluation orders for attributes of an SDD, dependency graphs. S-attributed and L-attributed SDDs and their implementation using LR-parsers and recursive-descent parsers respectively.                                                                                                                                                                                         | <b>15</b>  | <b>6</b>  |
| <b>5</b>     | <b>Semantic analysis</b><br>Symbol tables and their data structures. Representation of “scope”. Semantic analysis of expressions, assignment, and control-flow statements, declarations of variables and functions, function calls, etc., using S- and L-attributed SDDs (treatment of arrays and structures included). Semantic error recovery.                                                                                                                                                   | <b>15</b>  | <b>6</b>  |
| <b>6</b>     | <b>Intermediate code generation</b><br>Different intermediate representations –quadruples, triples, trees, flow graphs, SSA forms, and their uses. Translation of expressions (including array references with subscripts) and assignment statements. Translation of control-flow statements – if- then-else, while-do, and switch. Short-circuit code and control-flow translation of Boolean expressions. Back patching. Examples to illustrate intermediate code generation for all constructs. | <b>15</b>  | <b>6</b>  |
| <b>7</b>     | <b>Run-time environments</b> Stack allocation of space and activation records. Access to non-local data on the stack in the case of procedures with and without nesting of procedures.                                                                                                                                                                                                                                                                                                             | <b>10</b>  | <b>3</b>  |
| <b>8</b>     | <b>Introduction to machine code generation and optimization</b><br>Simple machine code generation, examples of machine-independent code optimizations.                                                                                                                                                                                                                                                                                                                                             | <b>5</b>   | <b>2</b>  |
| <b>Total</b> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>100</b> | <b>45</b> |



## Reference Books

- |    |                                                                                                                         |
|----|-------------------------------------------------------------------------------------------------------------------------|
| 1. | <b>Compilers: Principles, Techniques and Tools</b><br>By Aho, Lam, Sethi, and Ullman   Pearson   Second, Pub. Year 2014 |
|----|-------------------------------------------------------------------------------------------------------------------------|

## Course Outcome

**After Learning the Course the students shall be able to:**

- |    |                                                                                                                                                               |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. | Understand the basic concepts; ability to apply automata theory and knowledge on formal languages.                                                            |
| 2. | Ability to identify and select suitable parsing strategies for a compiler for various cases. Knowledge in alternative methods (top- down or bottom-up, etc.). |
| 3. | Understand backend of compiler: intermediate code, Code optimization Techniques and Error Recovery mechanisms                                                 |
| 4. | Understand issues of run time environments and scheduling for instruction level parallelism.                                                                  |